

Classificação do Dataset Iris utilizando Naive Bayes Gaussiano e Validação Cruzada Estratificada

Davi Israel Abtibol Carvalho

PGENE556 - Aprendizado de Máquina

Programa de Pós-Graduação em Engenharia Elétrica (PPGEE - UFAM)

Manaus, Amazonas, Brasil

davi.carvalho@ufam.edu.br

Resumo—Este trabalho apresenta a implementação e avaliação de um classificador Naive Bayes Gaussiano, desenvolvido integralmente do zero na linguagem Python, para a identificação de espécies de flores da base de dados Iris. Com o intuito de garantir uma avaliação robusta e aderente à distribuição original dos dados, foi implementado o método de Validação Cruzada com 5 pastas (5-Fold Cross-Validation) Estratificada, assegurando o balanceamento rigoroso das três classes presentes no dataset em partições de teste. Os resultados demonstraram a eficácia do modelo desenvolvido, alcançando uma acurácia média de 96,00% com um desvio padrão de apenas 1,33%.

Index Terms—Aprendizado de Máquina, Naive Bayes, Validação Cruzada, Iris Dataset, Estratificação.

I. INTRODUÇÃO

O problema de classificação multiclasse é um dos pilares do Aprendizado de Máquina supervisionado. Um dos conjuntos de dados mais clássicos e amplamente utilizados para introduzir e validar algoritmos de classificação é o *Iris Dataset*, originalmente publicado por Ronald Fisher em 1936 [1].

A base de dados é composta por 150 amostras de flores Íris, divididas perfeitamente de forma balanceada em três classes: *Setosa*, *Versicolor* e *Virgínica* (50 amostras cada). Para cada amostra, existem 4 características numéricas contínuas: comprimento e largura da sépala, e comprimento e largura da pétala.

O objetivo deste trabalho é relatar o desenvolvimento, desde sua base matemática até a implementação algorítmica, de um classificador Naive Bayes Gaussiano. Para fins de rigor metodológico, a implementação não faz uso de bibliotecas de alto nível prontas para Machine Learning (como o *scikit-learn*), apoiando-se apenas em ferramentas matemáticas fundamentais, como a biblioteca *NumPy* [2].

A avaliação do modelo é conduzida através do método de validação cruzada estratificada em 5 pastas (*5-Fold Stratified Cross-Validation*), atendendo ao requisito de garantir que todas as pastas contenham as mesmas proporções de classes encontradas no conjunto original.

II. METODOLOGIA

A. O Algoritmo Naive Bayes Gaussiano

O classificador Naive Bayes é embasado no Teorema de Bayes, fundamentando-se na premissa "ingênua" (naive) de que as características de um dado evento são condicionalmente independentes entre si. Como as características da base Iris são

compostas por valores contínuos (medidas em centímetros), é necessário assumir que esses valores seguem uma distribuição normal (Gaussiana).

Durante a fase de treinamento (*fit*), o algoritmo agrupa os dados de acordo com a classe c e, para cada uma das 4 características x_i , calcula a Média ($\mu_{c,i}$) e a Variância ($\sigma_{c,i}^2$).

Na fase de teste (*predict*), a probabilidade condicional de uma nova característica assumir determinado valor é dada pela Função de Densidade de Probabilidade (PDF) da distribuição normal:

$$P(x_i|c) = \frac{1}{\sqrt{2\pi\sigma_{c,i}^2}} \exp\left(-\frac{(x_i - \mu_{c,i})^2}{2\sigma_{c,i}^2}\right) \quad (1)$$

Para determinar a classe final da amostra, o modelo calcula a probabilidade *a posteriori* de cada classe e seleciona aquela com o maior valor. Assumindo a independência estatística entre as características (premissa central do Naive Bayes), a probabilidade conjunta do vetor de características é calculada através do produto de suas funções de densidade marginais. Dessa forma a regra de decisão consiste em classificar a amostra na classe c que maximiza a seguinte equação:

$$\text{Predição} = \arg \max_c \left(P(c) \prod_{i=1}^4 P(x_i|c) \right) \quad (2)$$

onde $P(c)$ é a probabilidade *a priori* da classe (neste caso, $1/3$ para as três classes perfeitamente balanceadas) e o produto envolve as probabilidades de cada uma das 4 características em relação à classe.

B. Separação dos Dados e Validação Cruzada

A separação pura e aleatória dos 150 exemplos em 5 pastas de 30 amostras não garante o balanceamento das classes na fase de avaliação. Para solucionar este problema, foi desenvolvida uma lógica de fatiamento estratificado (*Stratified K-Fold*).

O algoritmo executou os seguintes passos:

- 1) Separou os índices do conjunto de dados por classe (*Setosa*, *Versicolor*, *Virgínica*).
- 2) Aplicou uma permutação aleatória (embaralhamento) nos índices de cada classe, utilizando uma semente fixa ($\text{seed}=42$) para garantir a reprodutibilidade metodológica.

- 3) Fatiou as 50 amostras de cada classe em 5 blocos exatos de 10 amostras.
- 4) Para cada pasta k (de 1 a 5), o modelo agregou um bloco de 10 amostras de cada espécie, resultando em pastas de teste com exatamente 30 amostras, distribuídas proporcionalmente em 33,33% para cada classe.

O processo de validação cruzada consistiu em 5 iterações. Em cada iteração, uma pasta assumiu o papel de conjunto de teste (30 amostras), enquanto as 4 pastas restantes foram concatenadas para formar o conjunto de treinamento (120 amostras).

III. RESULTADOS E DISCUSSÕES

A implementação do classificador em conjunto com o particionamento estratificado obteve sucesso em sua compilação. As métricas extraídas nas cinco iterações de teste são apresentadas na Tabela I.

Tabela I
RESULTADOS DA VALIDAÇÃO CRUZADA (5-FOLD ESTRATIFICADO)

Rodada	Treino	Teste	Acertos	Acurácia (%)
1	120	30	29	96,67%
2	120	30	29	96,67%
3	120	30	29	96,67%
4	120	30	28	93,33%
5	120	30	29	96,67%
Acurácia Média Final			96,00%	
Desvio Padrão			$\pm 1,33\%$	

A acurácia média alcançada pelo modelo foi de **96,00%**, com um desvio padrão bastante reduzido de apenas **1,33%**. O baixo desvio padrão entre as rodadas é um reflexo direto do uso da estratificação. Como as pastas continham sempre a mesma proporção de dificuldades, o algoritmo teve um desempenho altamente estável e consistente, sem sofrer com vies de dados ou anomalias induzidas pelo particionamento aleatório desbalanceado.

Para compreender os 4% de erro do modelo, os resultados de todas as rodadas foram compilados em uma Matriz de Confusão Global (Tabela II).

Tabela II
MATRIZ DE CONFUSÃO GLOBAL (150 AMOSTRAS)

Real ↓ / Previsto →	Setosa	Versicolor	Virginica
Setosa	50	0	0
Versicolor	0	47	3
Virginica	0	3	47

A observação sistemática apontou que a falha de classificação ocorre exclusivamente entre instâncias limítrofes das classes *Versicolor* e *Virginica*. Conforme ilustrado na Figura 1, utilizando as medidas das pétalas (Medida 3 vs Medida 4), estas duas espécies compartilham interseções significativas em suas dimensões foliares, o que dificulta uma separação estatística perfeita.

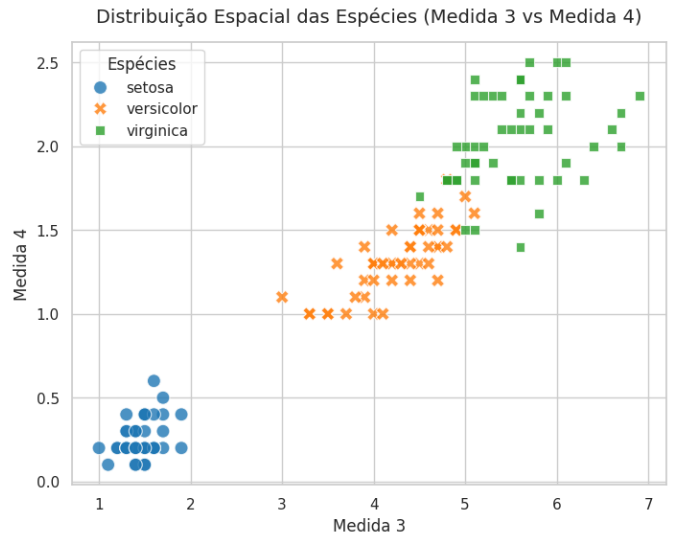


Figura 1. Distribuição espacial das espécies (Medida 3 vs Medida 4). Observa-se a zona de interseção entre Versicolor e Virginica.

Por outro lado, a figura também comprova que a espécie *Setosa* é dimensionalmente isolada das demais, o que corrobora a taxa de acerto sistemática de 100% do modelo para essa classe específica em todas as pastas de validação.

IV. CONCLUSÃO

O presente trabalho demonstrou na prática o funcionamento subjacente do aprendizado de máquina estatístico. Apesar de sua premissa central, de que todas as características são totalmente independentes, ser considerada "ingênuo". O Naive Bayes Gaussiano provou ser um método computacionalmente leve e excepcionalmente robusto para o *Iris dataset*.

Além disso, a implementação do método *Stratified K-Fold* direto nas matrizes de manipulação comprovou que um particionamento fidedigno aos dados originais é vital para certificar que a acurácia reportada é verdadeira e confiável, refletindo plenamente a real capacidade de generalização do modelo perante dados desconhecidos.

Por princípios de reprodutibilidade e transparência científica, o código-fonte completo deste trabalho, juntamente com o ambiente virtual documentado, encontra-se publicamente disponível no repositório do autor no GitHub através do link: <https://github.com/AbtibolDavi/aprendizado-maquina-2026>.

REFERÊNCIAS

- [1] R. A. Fisher, "Iris," UCI Machine Learning Repository, 1936, DOI: <https://doi.org/10.24432/C56C76>.
- [2] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020. [Online]. Available: <https://doi.org/10.1038/s41586-020-2649-2>